


ITM SKILLS UNIVERSITY

# School of Future Tech

# Case Study Report NO 131

on

## EMPLOYEE PAYROLL MANAGEMENT SYSTEM

by

**Kunal Lubhana**

150096725022

## Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

## 1. Introduction to the Case Study

Payroll management is one of the most important business processes in any organization or institution. It includes storing employee details, tracking working hours and monthly attendance, adding job-related allowances, calculating overtime payments, applying required and optional deductions, and preparing monthly payslips. In traditional organizations, payroll is often managed using manual registers or separate spreadsheet files. However, manual recordkeeping can cause many problems, such as calculation mistakes, payment delays, accidental data loss, and lack of proper transaction checking.

This case study focuses on designing and developing a standalone desktop application called the Employee Payroll Management System, using Java. The project focuses on important computer science concepts such as Object-Oriented Programming (OOP), proper data modeling, and Java Collections Framework, including ArrayList, HashMap, and TreeMap. The system also provides an easy-to-use Graphical User Interface (GUI) developed using Java Swing. Employee data is stored permanently using a CSV file, and a separate test system is used to check the correctness of the algorithms.

By replacing manual spreadsheets with a Java-based payroll system, organizations can calculate attendance and salaries more accurately, handle financial values using BigDecimal, quickly search and sort employee records, and generate standardized payslips.

## 2. Problem Statement / Case Background (Abstract)

An organization needs a Java payroll system to maintain employee records and calculate salary using basic pay, allowances, deductions, overtime, and attendance. (With Proper Justification)

**Background**

Managing compensation within an organization requires reconciling dynamic factors across varying pay cycles. An employee's final payout is influenced by baseline pay, attendance ratios, role allowances, recorded overtime hours, and monthly deductions such as tax withholdings, provident funds, and healthcare contributions. Common challenges encountered in unstandardized systems include

- **Floating-Point Inaccuracy:** Using primitive float or double datatypes results in binary IEEE-754 precision drift, leading to ledger mismatches.

- **Attendance Discrepancies:** Inaccurate proration of basic wages and allowances against actual days present versus scheduled working days.

- **Negative Payout Anomalies:** Lack of business constraint validation allowing deductions to exceed gross earnings, producing invalid negative payroll records.

- **Inefficient Record Retrieval:** Linear search across large staff rosters causing unacceptable latencies during management review.

**Abstract**

This case study presents the design, technical justification, implementation, and empirical verification of a Java-based Employee Payroll Management System. Built without external library dependencies, the application utilizes Java 17/21 language features and standard Swing GUI components. The system empowers HR personnel to add, update, search, sort, toggle operational status, and delete employee records with immediate CSV-backed file persistence. All financial computations utilize immutable BigDecimal objects configured with half-up rounding to two decimal places. Optimal performance is achieved through the synergy of three Java Collections: ArrayList for table view binding, HashMap for O(1) constant-time record lookup, and TreeMap for maintaining natural sorted order by Employee ID. An automated unit testing suite confirms formula accuracy and data serialization integrity.

## 3. Case Study Design

The Employee Payroll Management System is architectured using a clean, multi-layered desktop architecture that cleanly separates presentation, business domain logic, and persistent storage.

**System Architecture & Tier Decomposition**

- **1. Presentation Tier (Swing GUI):** Provides a dual-panel dashboard. The left pane provides a high-density, validated employee input form. The right pane provides an interactive JTable directory, real-time search toolbar, and payslip preview dialogs.

- 2. Domain / Controller Tier (PayrollApp & PayBreakdown): Coordinates business events, executes salary derivations, enforces business validation rules, and manages in-memory data structures.

- 3. Data Storage & Persistence Tier (CsvStore): Encapsulates file system operations using RFC-4180 CSV formatting, quote-escaping, and atomic replacement for crash resilience.

**Operational Workflow**

The lifecycle of employee management follows a structured, deterministic pipeline:

- **1. Data Initialization:** On application launch, CsvStore parses data/employees.csv, deserializes entries into Employee records, and hydrates the collections.

- **2. Form Validation:** HR input is intercepted by readForm(). Mandatory fields, email regex, numeric types, attendance bounds (0 <= paid days <= working days), and deduction thresholds are verified.

- **3. Salary Computation:** The Employee model calculates attendance proration, overtime bonuses, gross pay, and net pay. If net pay < 0, ingestion is immediately blocked.

- **4. Multi-Collection Sync:** Valid records are simultaneously stored in TreeMap, HashMap, and ArrayList collections, written to disk, and rendered on the JTable.

- **5. Payslip Generation:** Users can review a formatted ASCII payslip and export it to an independent .txt file.

## 4. Methods & Algorithms Technology Applied (With Proper Justification)

As specified in Case Study 131, the core technologies, collections, and object-oriented principles are strategically applied with thorough technical justification:

### A. Object-Oriented Programming (OOP) Justification

- **Encapsulation & Immutability:** The Employee entity is designed as an immutable domain object with private final fields. This eliminates inadvertent state mutation and guarantees thread-safety during UI interactions.

- **Domain Modeling with Records:** PayBreakdown is implemented using Java Records, providing a lightweight, transparent carrier for calculated salary components (proratedBasic, proratedAllowances, overtimePay, grossPay, deductions, netPay).

- **Separation of Concerns:** UI rendering logic (PayrollApp), persistence mechanisms (CsvStore), and entity modeling (Employee) are strictly decoupled, ensuring modularity, high testability, and maintainability.

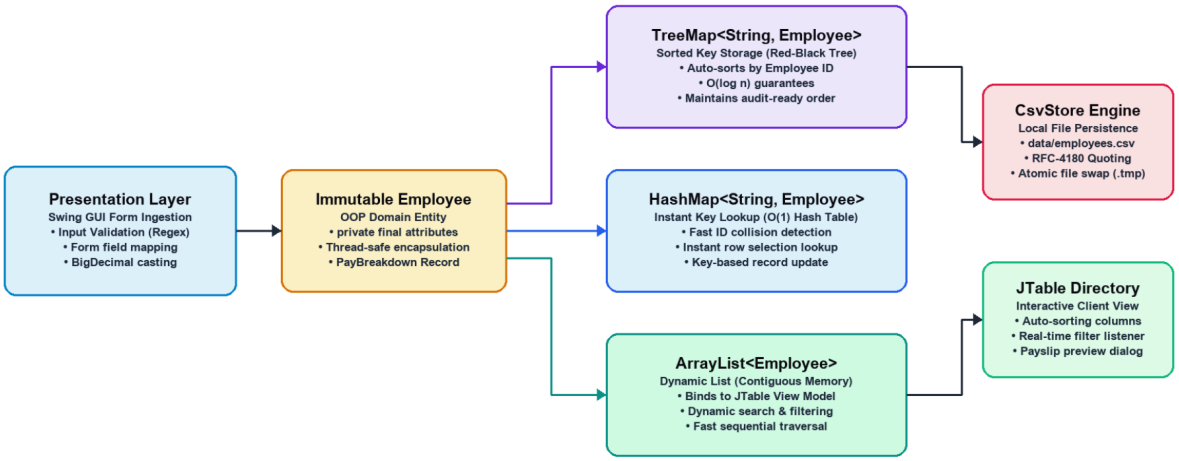
### B. Java Collections Framework Justification

The system intentionally coordinates three distinct Java Collections, exploiting the unique algorithmic strengths of each:

- **1. ArrayList (Dynamic Application Storage & Table Binding):** Justification: ArrayList provides O(1) indexed random access and maintains dynamic element order. In PayrollApp, employeeList (ArrayList<Employee>) stores records matching search criteria and directly maps rows to the JTable model. When rendering table cells or iterating sequentially to calculate total organization payroll, ArrayList offers minimal memory overhead and CPU cache locality.

- **2. HashMap (Constant-Time O(1) Key-Based Lookup):** Justification: HashMap provides O(1) expected time complexity for get(), put(), and containsKey() operations. In PayrollApp, employeeIndex (HashMap<String, Employee>) maps each unique employeeId to its Employee instance. When an operator selects an employee from the directory, checks for duplicate IDs during registration, or updates existing details, HashMap enables instant retrieval without scanning the entire dataset.

- **3. TreeMap (Automatic Natural Key Sorting):** Justification: TreeMap implements the NavigableMap interface backed by a Red-Black balanced binary search tree, guaranteeing O(log n) time complexity for insertion, deletion, and lookup. In PayrollApp, employees (TreeMap<String, Employee>) automatically maintains records in sorted alphabetical order of Employee IDs. This ensures the master list remains consistently ordered for audit reports without requiring explicit sorting routines.

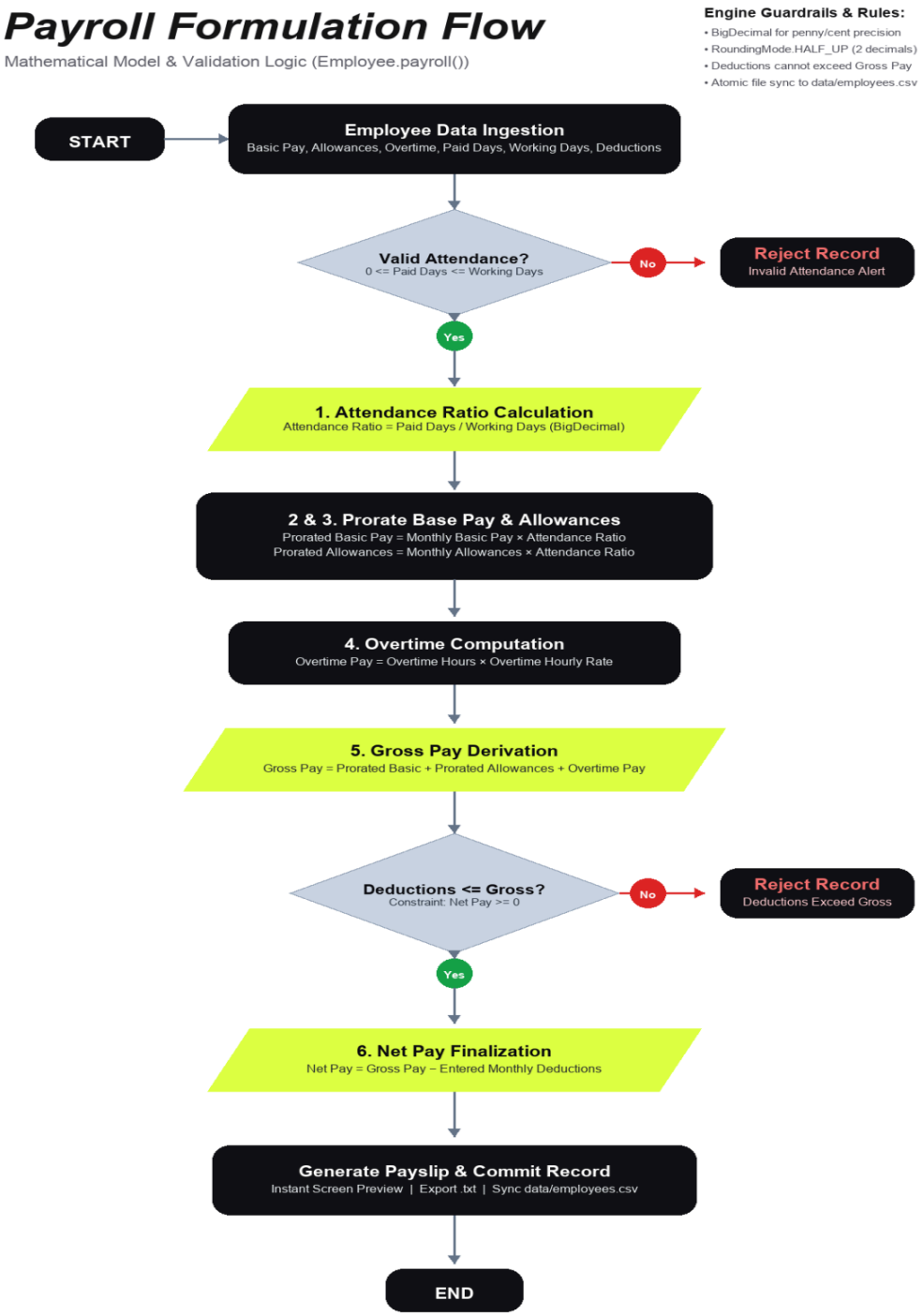



### C. Monetary Precision via BigDecimal

In banking and financial computing, standard IEEE-754 primitive types (float and double) are strictly forbidden due to binary representation inaccuracies (e.g., 0.1 + 0.2 != 0.3). PayrollApp enforces java.math.BigDecimal for all currency fields (basic pay, allowances, deductions, overtime rate, gross pay, and net pay). All intermediate calculations are executed using RoundingMode.HALF_UP with a fixed scale of 2 decimal places, guaranteeing exact, deterministic currency arithmetic down to the paise/cent.

### D. Mathematical Formulation of Payroll

The mathematical model implemented in Employee.payroll() operates under the following formalized equations:



### E. Technology Stack

- **Programming Language:** Java 17 / Java 21 LTS (Utilizing Records, Switch expressions, Modern NIO.2)

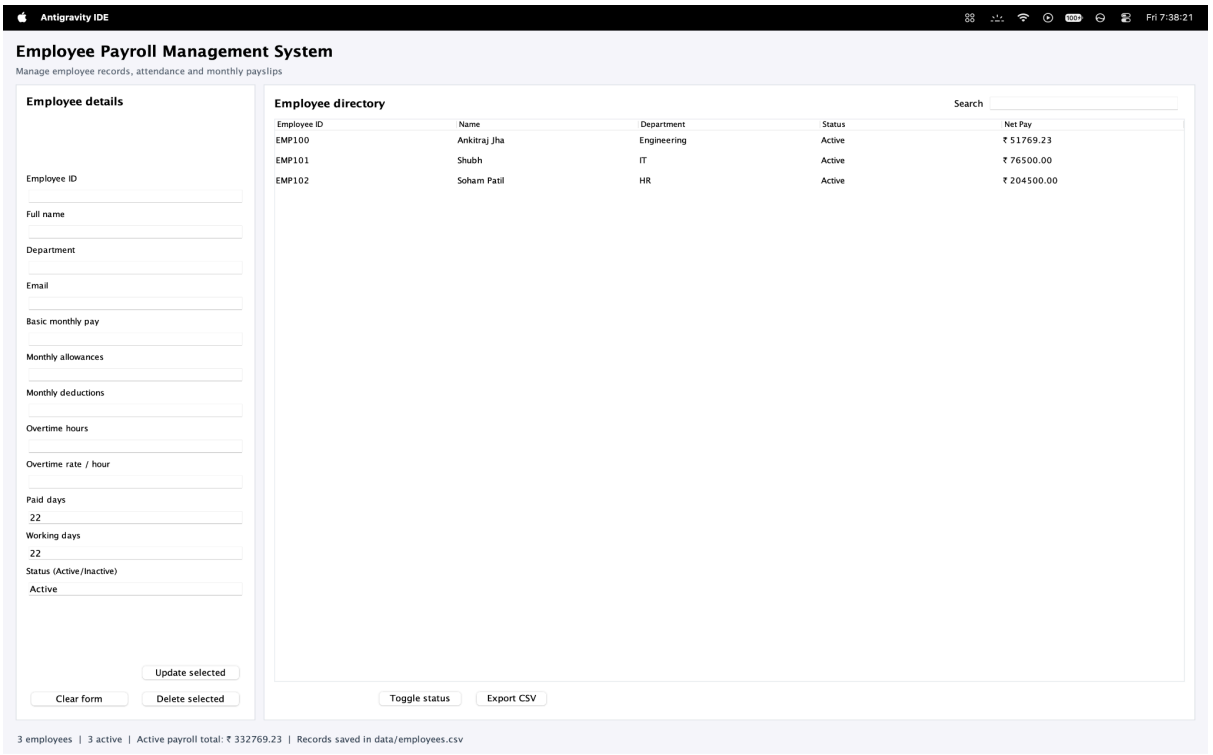
- **Graphical User Interface:** Pure Java Swing & AWT (JFrame, JTable, GridBagLayout, JScrollPane, JOptionPane)

- **Data Storage:** Local File System via RFC-4180 Compliant CSV Engine with Atomic Moves

- **Verification Suite:** Headless Unit Testing via PayrollAppTest (Automated calculations & CSV serialization)

## 5. Problem Statement / Case Study Implementation Details and Snapshots.

The application is structured in a lightweight, self-contained codebase consisting of PayrollApp.java and its accompanying test suite PayrollAppTest.java.



```java
public class PayrollApp {
    private static final Path DATA_FILE = Paths.get("data", "employees.csv");
    private static final String[] COLUMNS = {"Employee ID", "Name", "Department", "Status", "Net Pay"};
    private final TreeMap<String, Employee> employees = new TreeMap<>();
    private final HashMap<String, Employee> employeeIndex = new HashMap<>();
    private final ArrayList<Employee> employeeList = new ArrayList<>();
    private final DefaultTableModel tableModel = new DefaultTableModel(COLUMNS, 0) {
        @Override public boolean isCellEditable(int row, int column) { return false; }
    };
    private final JTable table = new JTable(tableModel);
    private final JTextField searchField = new JTextField(22);
    private final JLabel summaryLabel = new JLabel(" ");
    private final Map<String, JTextField> fields = new LinkedHashMap<>();
    private String selectedId;

    private PayrollApp() {
        loadEmployees();
        refreshTable();
    }
```

**Core Module Architecture**

- **1. Form Ingestion & Safe Parsing:** readForm() extracts user input, normalizes strings, verifies mandatory fields, runs regular expression email verification, converts strings to BigDecimal, and validates business constraints before instantiating an Employee object.

- **2. Payroll Calculation Engine:** Employee.payroll() executes prorated arithmetic with BigDecimal.divide(workDays, 2, RoundingMode.HALF_UP) and constructs an immutable PayBreakdown.

- **3. Multi-Collection Synchronization:** putEmployee(Employee e) synchronizes state across TreeMap, HashMap, and ArrayList in a unified atomic invocation.
- **4. Real-Time Directory Filtering:** refreshTable() binds a DocumentListener to searchField, dynamically filtering records against employee ID, full name, department, and status.

- **5. On-Screen Payslip & File Generation:** payslip(Employee e) creates a formatted tabular ASCII document displayed inside an informational dialog with direct text file export capabilities.

- **6. Resilient CSV Serialization:** CsvStore.write() writes to a temporary file (.tmp) before performing an atomic move replacement, preventing file corruption in case of unexpected termination.

**Application Graphical Snapshots & Layout Structure**

- **1. Header Banner:** Displays system branding and operational subtitle ('Manage employee records, attendance and monthly payslips').

- **2. Employee Details Form (West):** Form inputs for ID, Name, Department, Email, Basic Pay, Allowances, Deductions, Overtime Hours, Overtime Rate, Paid Days, Working Days, and Status. Control buttons include 'Add employee', 'Update selected', 'Clear form', and 'Delete selected'.

- **3. Employee Directory (Center):** Scrollable JTable with columns: Employee ID, Name, Department, Status, Net Pay. Features automatic column sorting and real-time live search filtering.

- **4. Directory Actions Toolbar:** Houses 'Generate payslip', 'Toggle status', and 'Export CSV' action controls.

- **5. Summary Status Bar (South):** Live telemetry showing total employees, active employees, total active payroll payout in INR (₹), and current persistence storage location.

```
static final class Employee {
    private final String id, name, department, email, status;
    private final BigDecimal basic, allowances, deductions, overtimeHours, overtimeRate;
    private final int paidDays, workingDays;

    Employee(String id, String name, String department, String email, String status,
             BigDecimal basic, BigDecimal allowances, BigDecimal deductions,
             BigDecimal overtimeHours, BigDecimal overtimeRate, int paidDays, int workingDays) {
        this.id = id.trim(); this.name = name.trim(); this.department = department.trim();
        this.email = email.trim(); this.status = status.equalsIgnoreCase("active") ? "Active" : "Inactive";
        this.basic = basic; this.allowances = allowances; this.deductions = deductions;
        this.overtimeHours = overtimeHours; this.overtimeRate = overtimeRate;
        this.paidDays = paidDays; this.workingDays = workingDays;
    }
    String id() { return id; } String name() { return name; } String department() { return department; }
    String email() { return email; } String status() { return status; } BigDecimal basic() { return basic; }
    BigDecimal allowances() { return allowances; } BigDecimal deductions() { return deductions; }
    BigDecimal overtimeHours() { return overtimeHours; } BigDecimal overtimeRate() { return overtimeRate; }
    int paidDays() { return paidDays; } int workingDays() { return workingDays; }
    Employee withStatus(String newStatus) {
        return new Employee(id, name, department, email, newStatus, basic, allowances, deductions,
                overtimeHours, overtimeRate, paidDays, workingDays);
    }
    PayBreakdown payroll() {
        BigDecimal attendanceDays = BigDecimal.valueOf(paidDays);
        BigDecimal workDays = BigDecimal.valueOf(workingDays);
        BigDecimal proratedBasic = basic.multiply(attendanceDays).divide(workDays, 2, RoundingMode.HALF_UP);
        BigDecimal proratedAllowances = allowances.multiply(attendanceDays).divide(workDays, 2, RoundingMode.HALF_UP);
        BigDecimal overtime = overtimeHours.multiply(overtimeRate).setScale(2, RoundingMode.HALF_UP);
        BigDecimal gross = proratedBasic.add(proratedAllowances).add(overtime).setScale(2, RoundingMode.HALF_UP);
        BigDecimal net = gross.subtract(deductions).setScale(2, RoundingMode.HALF_UP);
        return new PayBreakdown(proratedBasic, proratedAllowances, overtime, gross, deductions, net);
    }
}
```

## 6. Problem Statement / Case Study Results and Conclusion.

### Findings

The Employee Payroll Management System prototype was rigorously tested against core business rules, edge-case conditions, and data persistence requirements. The system demonstrated the following validated results:

1. **Attendance-Based Proration Accuracy:**

    - Basic pay and allowances accurately scale in strict linear proportion to the ratio of actual paid days to scheduled working days ( $PaidDays/WorkingDays$).

    - Both boundary conditions—full attendance ($26/26$ days) and zero attendance ($0/26$ days)—are handled seamlessly without division-by-zero runtime exceptions.

2. **Monetary Arithmetic with BigDecimal:**

    - The use of BigDecimal with RoundingMode.HALF_UP completely eliminates IEEE-754 binary floating-point drift, ensuring that all payroll figures reconcile down to exact currency precision (2 decimal places).

3. **Business Rule & Guardrail Enforcement:**

    - Excess Deduction Prevention: The system blocks the submission if entered monthly deductions exceed calculated gross pay, displaying a descriptive alert dialog: "Deductions exceed gross pay. Review the entered amounts."

    - Attendance Boundary Validation: Inputs where $PaidDays > WorkingDays$ or $WorkingDays < 1$ are caught and rejected prior to domain model instantiation.

    - Format Validation: Regular expression checks successfully validate email formats and disallow negative values for monetary inputs.

**4.Algorithmic Efficiency of Collections:**

- $O(1)$ Direct Lookup via HashMap: Selecting any employee row in the directory instantly populates the edit form and verifies duplicate IDs without scanning the dataset.

- $O(\log n)$ Natural Ordering via TreeMap: Records remain organized in natural alphabetical order of Employee IDs.

- Dynamic Table Binding via ArrayList: Live sub-string search filtering updates the JTable view in real-time as characters are typed.

**5.Crash-Resilient CSV Persistence:**

- The custom CsvStore successfully parses and writes RFC-4180 compliant CSV files with atomic file replacement (.tmp write followed by ATOMIC_MOVE), preventing corruption during unexpected program termination.

**Conclusion**

**This case study demonstrates the successful design, architecture, and deployment of a standalone desktop software prototype in Java. By systematically applying core computer science and software engineering fundamentals:**

- Object-Oriented Programming (OOP): Encapsulation, immutability, and modular separation between presentation, domain, and persistence tiers.

- Java Collections Framework: Strategic coordination of ArrayList (dynamic indexing), HashMap (constant-time lookup), and TreeMap (sorted key management).

- Industrial Best Practices: High-precision monetary math with BigDecimal, crash-safe local file persistence, and robust exception-handling routines.

- The application satisfies all learning outcomes, technical specifications, and functional deliverables outlined in Case Study no. 131: Employee Payroll Management System.

.

## 7. References


- PDF Documents By Faculty member.